

Módulo 9 – Capítulo 07 – Sección 01

Autenticación y autorización: OAuth2, API keys y control de acceso a modelos

Antes de que cualquier request llegue al modelo, debe responder dos preguntas independientes: ¿quién eres? (autenticación) y ¿qué puedes hacer? (autorización). En sistemas de IA, estas preguntas tienen dimensiones adicionales respecto a las APIs convencionales: la autorización debe ser granular no solo a nivel de endpoint sino a nivel de capacidad del modelo —qué modelos puede usar este cliente, con qué parámetros, en qué volumen—, y las credenciales de acceso no deben nunca aparecer en el contexto del LLM porque ese contexto puede ser extraído mediante prompt leaking. Comenzar el diseño de seguridad del endpoint por la autenticación no es un detalle de orden: sin autenticación, el rate limiting es irrelevante porque el atacante sin autenticar ya está en el punto incorrecto del sistema.

El stack de autenticación estándar para APIs de IA sigue los mismos protocolos que el software moderno: OAuth 2.0 con PKCE para aplicaciones que actúan en nombre de usuarios finales (donde el usuario debe autenticarse con su proveedor de identidad antes de que la aplicación pueda acceder al LLM en su nombre), API keys con rotación automática para comunicación server-to-server entre servicios internos, y JWT (JSON Web Tokens) con claims específicos de IA para autorización stateless en arquitecturas de microservicios. La diferencia con implementaciones convencionales es la granularidad de los claims en el JWT: además de los claims estándar (sub, iss, aud, exp), los claims específicos de IA deben incluir qué modelos están autorizados para esta identidad, qué límites de tokens aplican, y a qué tenants o recursos tiene acceso.

La jerarquía de API keys es el mecanismo de gestión de credenciales más utilizado en APIs de LLMs, y debe implementarse con tres niveles para minimizar el impacto de un compromiso: la clave maestra de organización —que no se usa directamente en ningún código de aplicación sino solo para generar claves de proyecto—, las project keys con scope limitado (qué modelos, qué endpoints, qué límites de tokens) que se usan en aplicaciones específicas, y los session tokens de corta duración que pueden generarse para requests específicas de alta sensibilidad.

Cada nivel tiene diferente capacidad de daño si se compromete y diferente frecuencia de rotación: la clave maestra rota anualmente con proceso controlado; las project keys rotan automáticamente cada 90 días; los session tokens expiran en horas o días.

El modelo de RBAC (Role-Based Access Control) para sistemas de IA traduce los permisos en términos de capacidades del modelo, no solo de endpoints. Un rol "basic" puede tener acceso a GPT-3.5-turbo con temperatura máxima de 0.7 y límite de 4k tokens por request; un rol "advanced" tiene acceso a GPT-4o con parámetros completos; un rol "fine-tuning-operator" tiene acceso a los endpoints de fine-tuning y a los modelos fine-tuned del tenant. Un rol "model-admin" puede ver y modificar los modelos fine-tuned de la organización. Esta granularidad no es un ejercicio académico: un atacante que compromete credenciales de un rol "basic" no puede acceder a los modelos fine-tuned ni a los endpoints de administración, limitando significativamente el daño potencial.

Aspectos técnicos

- **Jerarquía de API keys:** clave maestra (nunca en código, solo para generar project keys) → project keys con scope por modelo y endpoint (rotación automática 90 días) → session tokens de duración limitada (horas/días) para requests específicas; HashiCorp Vault con Dynamic Secrets para project keys y session tokens de duración limitada.
- **OAuth 2.0 con PKCE para usuarios finales:** el usuario se autentica con su IdP (Okta, Auth0, Azure AD) y la aplicación recibe tokens de acceso de alcance limitado al LLM; el modelo no recibe las credenciales del IdP, solo el token validado.
- **JWT con claims específicos de IA:** además de claims estándar, incluir `allowed_models[]`, `max_tokens_per_request`, `tenant_id`, `user_tier`; el middleware de autorización verifica estos claims antes de reenviar la request al proveedor del modelo.
- **RBAC por modelo y capacidad:** diferentes roles tienen acceso a diferentes modelos, endpoints, y parámetros; la autorización verifica que el `model_id` en la request está autorizado para el role del token de autenticación.
- **Detección de uso anómalo de API keys:** patrones de uso inusuales (nueva geolocalización, volumen anormal, endpoints no utilizados previamente) deben trigger alertas y opcionalmente suspensión temporal de la key para verificación; las credenciales comprometidas frecuentemente se usan primero con actividad anómala detectable.

Nota del Arquitecto: El error más común en autenticación de APIs de IA no es la ausencia de autenticación —casi todos los equipos la implementan— sino la ausencia de autorización granular. La misma API key que da acceso a las completions también

da acceso a los endpoints de fine-tuning, de listado de modelos, y de administración de la cuenta. Si esa key se filtra en un log de errores o en un repositorio público (que ocurre más frecuentemente de lo que los equipos reconocen), el atacante tiene acceso completo al sistema. La jerarquía de keys con scope limitado y rotación automática es la solución, y HashiCorp Vault la hace operacionalmente manejable.

La autenticación y autorización son el primer control que cualquier request debe atravesar, y su diseño granular es el prerrequisito para que todos los controles siguientes —rate limiting, input validation, output filtering— operen en el contexto correcto de quién está haciendo la request y qué está autorizado a hacer.

Módulo 9 – Capítulo 07 – Sección 02

Rate limiting y throttling: protección contra abuso y ataques de extracción masiva

Una vez que la identidad del cliente está verificada y sus permisos establecidos, el siguiente control en el ciclo de vida de la request es el rate limiting: cuánto puede acceder este cliente autenticado. En el contexto de sistemas de IA, el rate limiting tiene objetivos de seguridad que van más allá del rate limiting de APIs convencionales. Además de proteger contra ataques de DDoS y abuso de recursos computacionales, debe proteger contra model extraction —queries sistemáticas diseñadas para replicar el comportamiento del modelo—, contra ataques de amplificación donde el costo computacional de generar la respuesta supera significativamente el costo de la request (un prompt de 100 tokens puede generar una respuesta de 4,000 tokens con un uso de GPU desproporcionado), y contra ataques de context flooding que consumen toda la ventana de contexto disponible con inputs diseñados para agotar los recursos del sistema.

La granularidad del rate limiting es lo que determina su efectividad como control de seguridad. Un rate limit global por API key —por ejemplo, 1,000 requests por minuto— es insuficiente si diferentes endpoints tienen diferentes costos de seguridad y de recursos. El endpoint de completions tiene un perfil de riesgo diferente al endpoint de embeddings, que tiene un perfil diferente al de fine-tuning, que tiene un perfil diferente al de los endpoints de administración de modelos. El rate limiting correcto aplica límites independientes a cada categoría de endpoint, con valores calibrados al costo real de seguridad: el endpoint de fine-tuning, que puede iniciar un proceso de entrenamiento con costos de horas de GPU y acceso a todos los datos de entrenamiento del tenant, debe tener límites mucho más restrictivos que el endpoint de completions que procesa una sola request.

Las dimensiones del rate limiting para sistemas de IA deben cubrir al menos tres variables: requests por minuto (RPM), que protege contra ataques de alta frecuencia y DoS; tokens por minuto (TPM), que protege contra amplificación computacional y model extraction mediante queries largas; y ventanas temporales más largas (tokens por hora, tokens por día) para detectar ataques distribuidos de extracción que evaden los límites de corto plazo utilizando

patrones de baja frecuencia sostenida en el tiempo. La detección de model extraction requiere además análisis de la distribución de topics de los requests: un cliente que hace requests con alta diversidad semántica —cubriendo sistemáticamente el espacio de comportamientos del modelo en lugar de un caso de uso específico— exhibe el patrón característico de extracción, que debe trigger alertas incluso si cada request individual está dentro de los límites de RPM y TPM.

Para implementación técnica, Redis con Sliding Window Counter es el estándar de producción: una sorted set (ZSET) de Redis donde cada elemento es el timestamp de una request, permite contar requests en cualquier ventana temporal con una query ZRANGEBYSCORE sin la imprecisión del Fixed Window Counter que puede ser explotada ubicando requests en los bordes de las ventanas. Para TPM, la misma estructura acumula el count de tokens de cada request en lugar de contar requests unitariamente. La latencia de Redis para estas operaciones es <1ms en redes de datacenter, lo que la hace adecuada para el hot path de cada request.

Aspectos técnicos

- **Rate limiting multidimensional:** RPM (requests por minuto), TPM (tokens por minuto), y ventanas largas (tokens por hora/día) para detectar ataques distribuidos de baja frecuencia sostenida que evaden límites de corto plazo.
- **Límites específicos por endpoint:** endpoints de completions, embeddings, fine-tuning y administración de modelos con límites independientes calibrados al costo real de seguridad y recursos de cada operación; el endpoint de fine-tuning requiere límites mucho más restrictivos.
- **Algoritmos de implementación:** Sliding Window Counter con Redis ZSET para precisión sin vulnerabilidad a bordes de ventana fija; Token Bucket para casos de uso donde se permiten bursts hasta el tamaño del bucket; Leaky Bucket para traffic shaping uniforme.
- **Detección de model extraction en el rate limiter:** análisis de distribución de topics de los últimos N requests de cada API key usando embeddings; alta diversidad semántica que cubre sistemáticamente el espacio de comportamientos del modelo es señal de extracción.
- **Respuesta a rate limit excedido:** HTTP 429 con headers Retry-After y X-RateLimit-Remaining-Tokens como estándar; para patrones de extracción detectados, escalation a revisión manual y suspensión preventiva de la key.

Nota del Arquitecto: La trampa del rate limiting en IA es copiarlo directamente de los parámetros del proveedor: "OpenAI permite X requests por minuto en mi tier, así que configuro lo mismo". Los límites del proveedor protegen al proveedor de sus costos; los tuyos deben proteger tu sistema de sus riesgos específicos. Si un cliente de un rol "basic" puede hacer 1,000 requests por minuto al endpoint de fine-tuning, puede iniciar un proceso de entrenamiento masivo que consume tu presupuesto de GPU y potencialmente contamina el modelo con data poisoning en escala.

El rate limiting debe diseñarse como control de seguridad específico para los ataques de IA — model extraction, amplificación computacional, context flooding— no simplemente como protección de recursos computacionales, que es el único objetivo del rate limiting en APIs convencionales.

Módulo 9 – Capítulo 07 – Sección 03

Input validation: sanitización y límites de longitud antes de llegar al modelo

Con la identidad verificada y los límites de uso aplicados, la request autenticada llega al tercer control: la validación del contenido que va a enviarse al modelo. La validación de inputs en sistemas de IA tiene dos objetivos distintos que frecuentemente se confunden. El objetivo de seguridad busca detectar intentos de prompt injection, jailbreaking, o contenido diseñado para comprometer el sistema; el objetivo de calidad de servicio busca asegurar que los inputs están en el formato esperado, dentro de los límites operacionales, y son procesables por el modelo de manera eficiente. Confundir ambos objetivos lleva a implementaciones que mezclan controles con diferentes mecanismos y diferentes trade-offs de sensibilidad y performance.

La validación semántica del input —detectar intent malicioso en texto— no puede hacerse efectivamente mediante pattern matching basado en listas de palabras prohibidas o expresiones regulares. Esta estrategia, intuitiva pero fundamentalmente inadecuada, es eludible trivialmente: cualquier palabra prohibida puede reformularse mediante sinónimos, codificarse en Base64, traducirse a otro idioma, o fragmentarse en múltiples turns de conversación. Los clasificadores especializados son la herramienta correcta para la validación semántica. LlamaGuard-3, desarrollado por Meta y disponible en Hugging Face (llama-guard-3 en modelos de 8B parámetros), es un modelo fine-tuned específicamente para clasificar inputs y outputs contra 13 categorías de contenido dañino, incluyendo violencia, contenido sexual, actividades ilegales, y ciberseguridad ofensiva. Prompt Guard, también de Meta (2024), es un clasificador BERT-based diseñado específicamente para detectar intentos de prompt injection directa e indirecta, con latencia inferior a 10 milisegundos que lo hace viable en el hot path de cada request. Azure Content Safety API proporciona una alternativa gestionada para equipos que prefieren no operar infraestructura de clasificación propia.

La validación estructural complementa la semántica con controles sobre el formato del input. El límite de longitud en tokens —no en caracteres ni en bytes— es crítico porque el costo computacional del modelo escala con los tokens, y un input que es pequeño en caracteres

puede ser extremadamente largo en tokens para ciertos contenidos. El conteo de tokens debe hacerse con el tokenizador específico del modelo target: tiktoken para modelos OpenAI, el tokenizador de sentencepiece de Anthropic para Claude, el tokenizador de LLaMA para modelos Llama-3. El límite de longitud debe ser menor al context window del modelo para dejar espacio al system prompt, los documentos RAG, y la respuesta esperada; un input que consume todo el context window deja sin espacio para el system prompt y puede causar comportamientos inesperados.

La validación de schema es relevante cuando la aplicación acepta inputs estructurados: un formulario con campos específicos, un JSON con campos definidos, o una query en un formato controlado. En estos casos, la validación del schema del input antes de incluirlo en el prompt previene que el usuario añada campos no esperados que podrían alterar la estructura del prompt o inyectar instrucciones adicionales en el formato esperado.

Aspectos técnicos

- **Límites de longitud en tokens:** tokenizar el input con el tokenizador específico del modelo target antes de enviarlo; rechazar inputs que superen el límite definido (conservativamente menor al context window para dejar espacio a system prompt y respuesta); tiktoken para OpenAI, sentencepiece para Anthropic, llama-tokenizer para modelos Llama.
- **LlamaGuard-3 (Meta, open-source):** clasificador de 8B parámetros fine-tuned para 13 categorías de contenido dañino; latencia de 50-200ms dependiendo del hardware; disponible en Hugging Face como modelo open-source o via Fireworks AI como API gestionada.
- **Prompt Guard (Meta, 2024):** clasificador BERT-based para detección específica de prompt injection directa e indirecta; latencia <10ms apto para el hot path de cada request; distingue entre prompt injection directa (del usuario) e indirecta (en datos recuperados).
- **Validación de schema:** para inputs estructurados (JSON, formularios), validar el schema antes de incluir en el prompt; previene inyección de campos adicionales o modificación de la estructura esperada que podría alterar la interpretación del modelo.
- **Sanitización de documentos para ingestión RAG:** los documentos que ingresan al pipeline RAG deben pasar por las mismas validaciones que el input del usuario —un documento malicioso en el corpus es tan peligroso como un prompt malicioso en el input.

La validación de input no puede basarse únicamente en reglas de filtering de texto —son trivialmente eludibles— sino que debe combinar clasificadores semánticos especializados

(LlamaGuard, Prompt Guard) con controles estructurales (límites de tokens, validación de schema) que juntos forman la primera línea de defensa del contenido que llega al modelo.

Módulo 9 – Capítulo 07 – Sección 04

Output filtering: detección y redacción de contenido sensible en las respuestas

El output filtering es la capa de defensa que verifica las respuestas del modelo antes de entregarlas al usuario final. Su existencia como capa independiente del input filtering no es redundante: es la consecuencia lógica de aceptar que ningún control de input es perfecto y que el modelo puede generar outputs problemáticos incluso cuando el input parecía inofensivo. Los jailbreaks que el input filter no detectó, las alucinaciones que generan información médica incorrecta pero coherente, la PII memorada del corpus de pretraining que aparece en la respuesta sin que el usuario la haya mencionado, el texto del system prompt revelado mediante un ataque de prompt leaking exitoso —todos estos casos producen outputs problemáticos que llegan al usuario si no hay una segunda línea de defensa en el output.

La arquitectura de output filtering tiene múltiples capas que abordan diferentes tipos de problemas. LlamaGuard-3 ejecutado sobre el output del modelo clasifica la respuesta contra las mismas 13 categorías de contenido dañino que el input filter, proporcionando una segunda oportunidad de detección para jailbreaks que el modelo ejecutó a pesar del input filter. La detección de PII en el output —ejecutando Presidio sobre la respuesta antes de entregarla— detecta datos personales que el modelo pudo haber incluido provenientes de memorización del pretraining, del contexto del usuario incluido de maneras no esperadas, o de documentos RAG que contenían PII no redactada. La detección del texto del system prompt en la respuesta —mediante búsqueda de n-gramas del system prompt en el output— detecta intentos de prompt leaking exitosos donde el modelo incluyó el contenido del system prompt en su respuesta.

La validación de políticas específicas de la aplicación es la dimensión del output filtering más subestimada. Las políticas de contenido del proveedor del modelo —OpenAI Content Policy, Anthropic Usage Policy— son el mínimo regulatorio del proveedor, no el máximo de la aplicación. Una aplicación de atención al cliente de un banco tiene restricciones adicionales que el LLM base no puede conocer: no generar consejo de inversión específico, no mencionar productos de la competencia, no revelar información que no está en los documentos

autorizados del banco, no hacer promesas sobre plazos o condiciones que el banco no puede garantizar. Estas políticas específicas de la aplicación deben verificarse en el output antes de entregarlo al usuario, independientemente de si el proveedor del modelo las implementa o no.

El streaming es una consideración adicional en el diseño del output filter. Cuando el sistema usa streaming —el output llega token a token, mejorando la experiencia percibida del usuario— el output filtering debe operar en modo buffered (acumular el output completo antes de entregarlo) para poder clasificarlo como un todo, o en modo chunk-based (evaluar chunks de tokens acumulados), con un trade-off explícito entre latencia percibida y cobertura de detección. En modo buffered, el usuario espera hasta que el output completo ha sido validado antes de ver la primera palabra; en modo chunk-based, el usuario ve chunks validados con latencia más baja pero con menor cobertura ante ataques distribuidos en el output.

Aspectos técnicos

- **LlamaGuard sobre el output:** ejecutar LlamaGuard-3 sobre la respuesta del modelo antes de entregarla al usuario; si el output clasifica como violación de alguna de las 13 categorías, retornar un mensaje de error estándar sin revelar qué categoría fue violada.
- **Detección de PII en outputs con Presidio:** escanear el output del modelo con Microsoft Presidio para detectar y redactar PII que el modelo pudo incluir de memorización del pretraining, del contexto del usuario, o de documentos RAG no correctamente anonimizados.
- **Detección del system prompt en el output:** búsqueda de n-gramas del system prompt en el output para detectar prompt leaking exitoso; alternatively, comparar fragmentos del output contra el hash del system prompt para detectar similitudes significativas.
- **Content policy enforcement específico de la aplicación:** verificación de restricciones de contenido que van más allá de las políticas del proveedor del modelo: no consejo de inversión, no promesas de plazos, no mención de competidores, no información no autorizada por la política editorial del sistema.
- **Streaming y output filtering:** modo buffered (menor latencia para el output filter, mayor latencia percibida para el usuario) vs. modo chunk-based (menor latencia percibida, menor cobertura de detección); la elección debe documentarse explícitamente con su justificación.

Nota del Arquitecto: El output filtering añade latencia — típicamente 50-200ms para LlamaGuard, más para Presidio sobre responses largas. He visto equipos

eliminarlo en producción porque "ralentiza la experiencia del usuario". La decisión correcta no es eliminarlo sino optimizarlo: LlamaGuard puede ejecutarse en paralelo con el streaming del output para que el output se presente al usuario solo si la evaluación en paralelo no detecta violaciones; Presidio puede ejecutarse solo sobre outputs que superen un threshold de longitud. El output filter no puede ser opcional porque es la última línea de defensa.

El output filtering es la última línea de defensa antes de que el contenido del modelo llegue al usuario: siempre debe implementarse, independientemente de cuán robusto sea el input filtering, porque no existe sistema de input filtering con cero falsos negativos.

Módulo 9 – Capítulo 07 – Sección 05

Web Application Firewall adaptado para APIs de IA

Los Web Application Firewalls tradicionales —AWS WAF, Cloudflare WAF, ModSecurity— protegen aplicaciones web contra ataques cuya firma está en la estructura de la request HTTP: SQL injection en parámetros de URL, XSS en campos de formulario, path traversal en rutas, CSRF en cookies. Para APIs de IA, esta protección de capa HTTP es necesaria pero radicalmente insuficiente: un prompt injection no tiene firma a nivel de protocolo HTTP porque el payload malicioso es texto semánticamente válido en el campo `message.content` del JSON. El WAF tradicional ve una request POST a `/v1/chat/completions` con JSON bien formado y la pasa sin alarma, sin capacidad de detectar que el texto de ese JSON es un jailbreak o una prompt injection sofisticada.

Un WAF para APIs de IA debe operar en dos capas independientes que se complementan. La capa HTTP clásica —AWS WAF, Cloudflare WAF— proporciona protección perimetral convencional: rate limiting de primera línea por IP, bloqueo de rangos de IPs maliciosas según listas de threat intel, protección contra bots, y defensa ante ataques de capa HTTP convencionales que también pueden intentarse contra endpoints de IA. Esta capa no puede eliminarse porque los atacantes no solo usan técnicas específicas de IA; también prueban técnicas convencionales contra cualquier endpoint expuesto. La capa semántica especializada para IA complementa la capa HTTP con capacidades que esta no puede proporcionar: análisis del contenido del prompt para detectar injection, tracking de patrones de uso a lo largo de múltiples sessions para detectar extracción sistemática, y análisis de riesgo basado en la semántica del input.

En el ecosistema de WAF semántico para IA destacan tres soluciones con diferentes trade-offs. Lakera Guard es un proxy inverso para APIs de LLMs que intercepta requests y responses, aplica clasificadores de prompt injection y jailbreak entrenados en un dataset propietario, y proporciona scores de riesgo por request; su arquitectura de sidecar no requiere modificar el código de la aplicación, lo que facilita la adopción. Prompt Security tiene un enfoque similar pero añade análisis de comportamiento entre sessions y shadow mode — logging de todos los prompts sin bloqueo para análisis forense antes de activar el modo de

bloqueo en producción. Rebuff es la alternativa open-source: una librería Python que combina tres capas de detección (heurísticas de reglas rápidas, LLM-based detection con un modelo secundario que evalúa si el input contiene injection, y VectorDB-based detection que compara el embedding del input contra una base de ejemplos conocidos de injection), integrable directamente en la aplicación sin proxy externo.

Los equipos que operan en entornos self-hosted o on-premises —sin acceso a servicios cloud específicos como AWS WAF o Cloudflare— deben considerar ModSecurity con el ruleset de OWASP Core Rule Set para la capa HTTP, y Rebuff o una implementación personalizada de LlamaGuard como capa semántica. OPA (Open Policy Agent) puede complementar el WAF con políticas de autorización granular basadas en el contenido de la request, implementando reglas como "usuarios del role basic no pueden enviar requests con contextos de más de X tokens" o "este endpoint solo es accesible desde rangos de IP de la red corporativa".

Aspectos técnicos

- **WAF tradicional para la capa HTTP:** AWS WAF, Cloudflare WAF, o ModSecurity con OWASP Core Rule Set para ataques convencionales (rate limiting por IP, bots, ataques de capa HTTP); necesario pero insuficiente para detectar ataques específicos de LLMs.
- **Lakera Guard:** proxy inverso de sidecar para APIs de LLMs; clasificadores de prompt injection y jailbreak con dataset propietario; scores de riesgo por request; soporta OpenAI, Anthropic y endpoints de modelos propios; no requiere modificar código de aplicación.
- **Prompt Security:** similar a Lakera con análisis de comportamiento entre sessions; shadow mode para logging sin bloqueo durante la fase de calibración antes de activar el modo restrictivo en producción.
- **Rebuff (open-source):** tres capas de detección combinadas (heurísticas, LLM-as-detector, VectorDB similarity search); integrable como librería Python directamente en la aplicación; adecuado para entornos self-hosted donde los proxies externos no son viables.
- **Alternativas on-premises:** ModSecurity (OWASP CRS) para la capa HTTP + Rebuff o LlamaGuard embebido para la capa semántica + OPA para políticas de autorización granular basadas en el contenido de la request.

Un WAF para APIs de IA debe operar en las dos capas independientemente y en paralelo: la capa HTTP clásica para protección perimetral convencional, y la capa semántica especializada para detectar ataques específicos de LLMs que no tienen firma a nivel de protocolo HTTP.

Módulo 9 – Capítulo 07 – Sección 06

Cierre: el hardening del endpoint es la primera línea de defensa del sistema de IA

Las cinco capas del hardening del endpoint —autenticación y autorización, rate limiting, input validation, output filtering, y WAF— no son controles independientes que se eligen según el presupuesto o la conveniencia; son las capas de un filtro que cada request debe atravesar en el orden específico que refleja el ciclo de vida de la request. La autenticación y autorización primero porque sin ellas el resto carece de contexto. El rate limiting segundo porque limita el volumen antes de que los controles más costosos (clasificadores semánticos) deban procesar un ataque de alta frecuencia. El input validation tercero porque filtra el contenido antes de que llegue al modelo. El output validation cuarto porque es la red de seguridad después del modelo. El WAF como capa perimetral que opera en paralelo con el ciclo de vida interno. Reordenar estas capas o eliminar alguna no equivale a elegir un nivel de seguridad diferente; equivale a dejar brechas estructurales que los atacantes explotan sistemáticamente.

El hardening del endpoint es la primera línea de defensa del sistema, pero tiene una limitación que los equipos maduros reconocen explícitamente: no previene todos los ataques. Los ataques de data poisoning que ocurren en el pipeline de entrenamiento operan por debajo del endpoint. Los ataques de model extraction distribuidos que se mantienen por debajo de los umbrales de rate limiting pueden ser invisibles durante semanas. Los ataques adversariales en texto contra los clasificadores-guardrail pueden eludir el input filter. La primera línea de defensa robusta eleva significativamente el costo de cualquier ataque exitoso, pero no reemplaza las capas de defensa siguientes —sandboxing, autorización granular de herramientas, logging e inmutabilidad de auditoría— que son necesarias para contener el daño cuando la primera línea es superada.

En términos prácticos, la mayoría de los intentos de abuso de sistemas de IA en producción son oportunistas, no sofisticados: bots que intentan usar la API sin autenticación, usuarios que prueban jailbreaks conocidos que circulan en comunidades online, competidores que intentan consultas sistemáticas de baja frecuencia para replicar el comportamiento del sistema. Un hardening correcto de la primera línea —autenticación, rate limiting calibrado,

LlamaGuard en input y output— es suficiente para neutralizar la gran mayoría de estos intentos. Los ataques sofisticados de actores con alta motivación y capacidades avanzadas requieren defensa en profundidad que va más allá del endpoint, pero esa defensa en profundidad no es efectiva sin una primera línea robusta.

Conceptos clave del capítulo

- **Autenticación y autorización granular:** OAuth 2.0 / API keys con jerarquía de tres niveles, RBAC por modelo y capacidad, JWT con claims específicos de IA, secrets en HashiCorp Vault nunca en código ni contexto del modelo.
- **Rate limiting multidimensional:** RPM + TPM + ventanas largas para detección de extracción distribuida; límites independientes por categoría de endpoint; Sliding Window Counter en Redis para producción; detección de model extraction mediante análisis de distribución de topics.
- **Input validation semántica:** límites de longitud en tokens con el tokenizador específico del modelo, LlamaGuard-3 para clasificación semántica, Prompt Guard para detección específica de injection, validación de schema para inputs estructurados.
- **Output filtering:** LlamaGuard sobre el output como segunda línea de defensa, Presidio para redacción de PII, detección del system prompt en la respuesta, enforcement de policies específicas de la aplicación, consideraciones de streaming.
- **WAF dual-layer:** capa HTTP (AWS WAF, Cloudflare, ModSecurity) para ataques perimetrales convencionales + capa semántica (Lakera Guard, Prompt Security, Rebuff) para ataques específicos de LLMs; alternativas on-premises disponibles para entornos self-hosted.

"Defense is difficult not because attackers are smart, but because defenders must be right every time while attackers only need to be right once." — John Chambers, ex CEO de Cisco, sobre el principio de asimetría fundamental entre atacantes y defensores que hace al hardening de la primera línea crítico: cuanto más costoso sea el único intento exitoso del atacante, más efectiva es la primera línea de defensa.